

```
CREATE TABLE IF NOT EXISTS pessoa (  
    id_pessoa INTEGER PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    nascimento DATE  
);
```

```
CREATE TABLE IF NOT EXISTS carro (  
    id_carro INTEGER PRIMARY KEY,  
    placa CHAR(7) NOT NULL,  
    ano INTEGER,  
    id_pessoa INTEGER NOT NULL,  
    FOREIGN KEY (id_pessoa) REFERENCES pessoa (id_pessoa) ON DELETE CASCADE  
);
```

```
COPY pessoa (id_pessoa, nome, nascimento)  
FROM 'C:\Users\User\Desktop\FATEC-JCR-2DSM-BDR-2026-1\P2-Conteudos\Atividades\Atividade-  
Aula12/aula3_pessoa.csv'  
DELIMITER ','  
CSV HEADER;
```

```
COPY carro (id_carro, placa, ano, id_pessoa)  
FROM 'C:\Users\User\Desktop\FATEC-JCR-2DSM-BDR-2026-1\P2-Conteudos\Atividades\Atividade-  
Aula12/aula3_carro.csv'  
DELIMITER ','  
CSV HEADER;
```

```
-- EX 1 PT A
```

```
EXPLAIN ANALYZE  
SELECT *  
FROM pessoa  
WHERE nome = 'João Santos';
```

-- PT B

```
CREATE INDEX idx_pessoa_nome  
ON pessoa (nome);
```

-- PT C

```
EXPLAIN ANALYZE  
SELECT *  
FROM pessoa  
WHERE nome = 'João Santos';
```

-- PT D

-- Qual plano foi utilizado antes? Seq Scan (Varredura Sequencial), pois o banco teve que ler todas as 100 mil linhas .

-- Qual plano foi utilizado após? Index Scan.

-- Houve redução no tempo? Sim, uma redução drástica (geralmente cai de alguns milissegundos para frações de milissegundo).

-- Foi utilizado em ambos os nomes? Sim, o índice B-tree atende a qualquer busca por igualdade nessa coluna.

-- EX 2 PT A

```
DROP INDEX IF EXISTS idx_pessoa_nome;
```

```
EXPLAIN ANALYZE  
SELECT *  
FROM pessoa  
WHERE nascimento >= DATE '1970-01-01'
```

-- PT B

```
CREATE INDEX idx_pessoa_nascimento
```

```
ON pessoa (nascimento);
```

```
-- PT C
```

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento >= DATE '1970-01-01'
```

```
-- PT D
```

```
-- O índice foi utilizado? Geralmente Não.
```

```
-- O plano mudou? Não, continuou sendo um Seq Scan.
```

```
-- Houve melhora? Não.
```

```
-- Por que o otimizador ignorou o índice? Por causa da baixa seletividade.
```

```
-- A maioria das pessoas no banco nasceu depois de 1970. Como a consulta retorna a maior parte da  
tabela, o otimizador acha mais barato ler tudo de uma vez do que usar o índice .
```

```
-- EX 3
```

```
DROP INDEX IF EXISTS idx_pessoa_nascimento;
```

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento >= DATE '2000-01-01'
```

```
AND nome = 'Ana Silva';
```

```
-- PT B
```

```
CREATE INDEX idx_pessoa_nascimento
```

ON pessoa (nascimento, nome);

-- PT D

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nome = 'Ana Silva';

-- PT E

-- Plano antes: Seq Scan.

-- Plano após: Index Scan / Bitmap Index Scan.

-- Foi utilizado na consulta apenas por nome? Não.

-- Por que a ordem é relevante? Porque o índice funciona como uma lista telefônica. Se o índice é (nascimento, nome), ele está ordenado primeiro pelas datas. É impossível buscar alguém só pelo "nome" nesse índice sem ler ele inteiro .

-- EX 4 PT A

]

DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;

DROP INDEX IF EXISTS idx_pessoa_nascimento;

CREATE INDEX idx_pessoa_nascimento

ON pessoa (nascimento);

CREATE INDEX idx_pessoa_nome

ON pessoa (nome);

-- PT B

EXPLAIN ANALYZE

SELECT *

FROM pessoa

WHERE nascimento >= DATE '2000-01-01'

AND nome = 'Ana Silva';

-- PT C

-- O PostgreSQL utilizou os dois índices simples na consulta? Sim.

-- Qual é o papel do operador BitmapAnd no plano de execução? Ele cruza (faz a interseção / o "AND") dos resultados encontrados nos dois índices diferentes diretamente na memória.

-- O que o plano efetivamente fez? Ele achou os ponteiros de quem nasceu após 2000 em um índice, achou os ponteiros das 'Ana Silva' no outro, comparou as duas listas e só foi na tabela buscar os registros que estavam nas duas listas.

-- EX 5

DROP INDEX IF EXISTS idx_pessoa_nascimento;

DROP INDEX IF EXISTS idx_pessoa_nome;

-- 1. Antes de criar o índice (Para registrar o plano original - Seq Scan)

EXPLAIN ANALYZE

SELECT *

FROM carro

WHERE ano BETWEEN 2015 AND 2020;

-- 2. Criando o índice mais adequado para o intervalo

CREATE INDEX idx_carro_ano ON carro(ano);

-- 3. Depois de criar o índice (Compara o plano)

EXPLAIN ANALYZE

SELECT *

FROM carro

WHERE ano BETWEEN 2015 AND 2020;

-- EX 6

DROP INDEX IF EXISTS idx_carro_ano;

-- 1. Antes de criar os índices

EXPLAIN ANALYZE

SELECT p.nome, c.placa

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nome = 'Ana Silva';

-- 2. Criando os índices necessários para otimizar o JOIN

CREATE INDEX idx_pessoa_nome_join ON pessoa(nome);

CREATE INDEX idx_carro_fk_pessoa ON carro(id_pessoa);

-- 3. Depois de criar os índices

EXPLAIN ANALYZE

SELECT p.nome, c.placa

FROM pessoa p

JOIN carro c ON p.id_pessoa = c.id_pessoa

WHERE p.nome = 'Ana Silva';

-- EX 7

DROP INDEX IF EXISTS idx_pessoa_nome_join;

DROP INDEX IF EXISTS idx_carro_fk_pessoa;

-- 1. Antes de criar os índices

EXPLAIN ANALYZE

SELECT p.nome, c.placa, c.ano

FROM pessoa p

```
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nascimento >= DATE '1980-01-01'
AND c.ano >= 2018;
```

-- 2. Criando os índices necessários

```
CREATE INDEX idx_pessoa_nascimento_join ON pessoa(nascimento);
CREATE INDEX idx_carro_composto_join ON carro(id_pessoa, ano);
```

-- 3. Depois de criar os índices

```
EXPLAIN ANALYZE
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nascimento >= DATE '1980-01-01'
AND c.ano >= 2018;
```

-- Justificativa da escolha: Para a tabela 'pessoa', usei um índice simples porque ela só sofre um filtro (nascimento).

-- Já para a tabela 'carro', usei um índice COMPOSTO porque ela é usada simultaneamente no JOIN (id_pessoa) e no filtro (ano).

-- EX 8 PT A

```
DROP INDEX IF EXISTS idx_pessoa_nascimento_join;
DROP INDEX IF EXISTS idx_carro_composto_join;
```

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

-- PT B

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

```
CREATE INDEX idx_pessoa_nascimento_gist  
ON pessoa USING GIST (nascimento);
```

```
SELECT indexname, indexdef  
FROM pg_indexes  
WHERE tablename = 'pessoa';
```

-- PT C

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM pessoa
```

```
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

-- PT D

-- Qual plano foi utilizado antes da criação do índice GIST? Seq Scan (Varredura Sequencial).

-- O índice GIST foi utilizado após a criação? Sim, o plano mudou para utilizar o índice GIST (Bitmap Index Scan).

-- Houve melhora no tempo de execução? Sim, buscas de intervalo com muitos registros são otimizadas eficientemente pelo GIST.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including 'aula12_indices' and 'aula12_08'. The main window shows a SQL query in the 'Query' tab, which is the same query as shown in the text blocks above. The 'Query History' tab shows the query was executed at 18:18. The 'Data Output' tab shows the query plan, which is a 'Bitmap Index Scan on idx_pessoa_nascimento'. The plan details include: 'Bitmap Heap Scan on pessoa (cost=265.01..1188.21 rows=15680 width=23) (actual time=0.811..5.666 rows=15745.00 loops=1)', 'Recheck Cond: ((nascimento >= '1980-01-01'::date) AND (nascimento <= '1990-12-31'::date))', 'Heap Blocks: exact=665', 'Buffers: shared hit=692', and 'Bitmap Index Scan on idx_pessoa_nascimento (cost=0.00..261.09 rows=15680 width=0) (actual time=0.693..0.693 rows=15745.00 loops=1)'. The bottom status bar shows 'Showing rows: 1 to 12' and 'Page No: 1 of 1'.